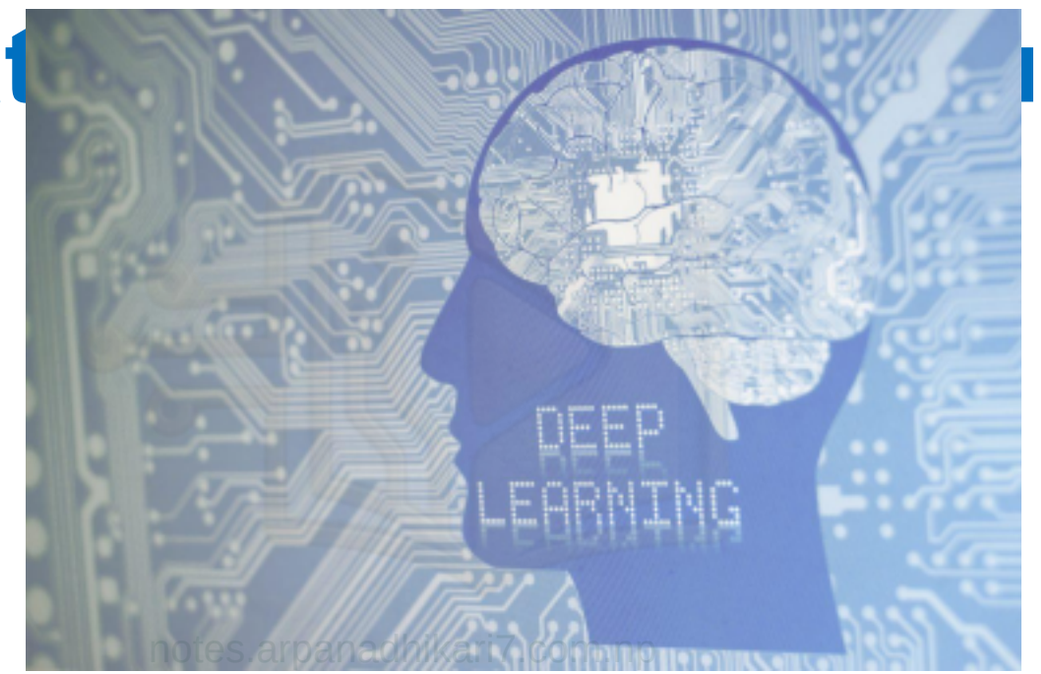
Generative Artificial Intelligence

Unit 2: Foundat[illegible]age Models



notes.arpanadhikari7.com.np

Basanta Joshi, PhD

Assistant Professor, Department of Electronics and Computer Engineering,

Pulchowk Campus

Institute of Engineering, Tribhuvan University

basanta@ioe.edu.np

http://www.basantajoshi.com.np

https://scholar.google.com/citations?user=iocLiGcAAAAJ

https://www.researchgate.net/profile/Basanta_Joshi2

Available on notes.arpanadhikari7.com.np

Regarding Course

- Chapter 1: Introduction to Gen AI
  - — Theory: Introduction Slides
  - — Practical : Python, numpy, pandas, FastAPI :
  - — Lab Exercise 1
- Chapter 2: Foundation of LLM
  - — Theory: Deep Learning → Transformer
  - — Tokenization, BPE, Word2Vec, Document Embeddings:
  - — Lecture slide NN+ this slide+”NLP”
- Chapter 3: Prompt Engineering
  - — Theory: Prompt Engineering course + Assignment (https://www.deeplearning.ai/courses/chatgpt-prompt-eng
  -) — Practical: Lab Exercise LLM Integration

notes.arpanadhikari7.com.np

- Chapter 4: LangChain
  - Practical: Next Lab Exercise
- Chapter 5: RAG
  - Theory: Lecture presentation “Agentic AI”
  - Practical: Lab Exercise LLM Integration + Langchain Lab exercise
- Chapter 6, Chapter 7:Self Study

Regarding Course

notes.arpanadhikari7.com.np

- Chapter 1: Introduction to Gen AI
  - Theory: Introduction Slides
  - Practical : Python, numpy, pandas, FastAPI :
  - Lab Exercise 1
- Chapter 2: Foundation of LLM
  - Theory: Deep Learning→ Transformer
  - Tokenization, BPE, Word2Vec, Document Embeddings:

- Lecture slide NN+ this slide+”NLP”

- Chapter 3: Prompt Engineering
  - Theory: Prompt Engineering course + Assignment (https://www.deeplearning.ai/courses/chatgpt-prompt-eng) – Practical: Lab Exercise LLM Integration
- Chapter 4: LangChain
  - Practical: Next Lab Exercise
- Chapter 5: RAG
  - Theory: Lecture presentation “Agentic AI”
  - Practical: Lab Exercise LLM Integration + Langchain Lab exercise
- Chapter 6, Chapter 7:Self Study

Slide | 3 By: Basanta Joshi, PhD
notes.arpanadhikari7.com.np

What is Large Language Model?

- An LLM, a large language model, is a neural network designed to understand, generate, and respond to human-like text

Available on notes.arpanadhikari7.com.np

- These models are deep neural networks trained on massive amounts of text data, sometimes encompassing large portions of the entire publicly available text on the internet.

- The "large" in large language model refers to both the model's size in terms of parameters and the immense dataset on which it's trained • Models like this often have tens or even hundreds of billions of parameters, which are the adjustable weights in the network that are optimized during training to predict the next word in a sequence

notes.arpanadhikari7.com.np

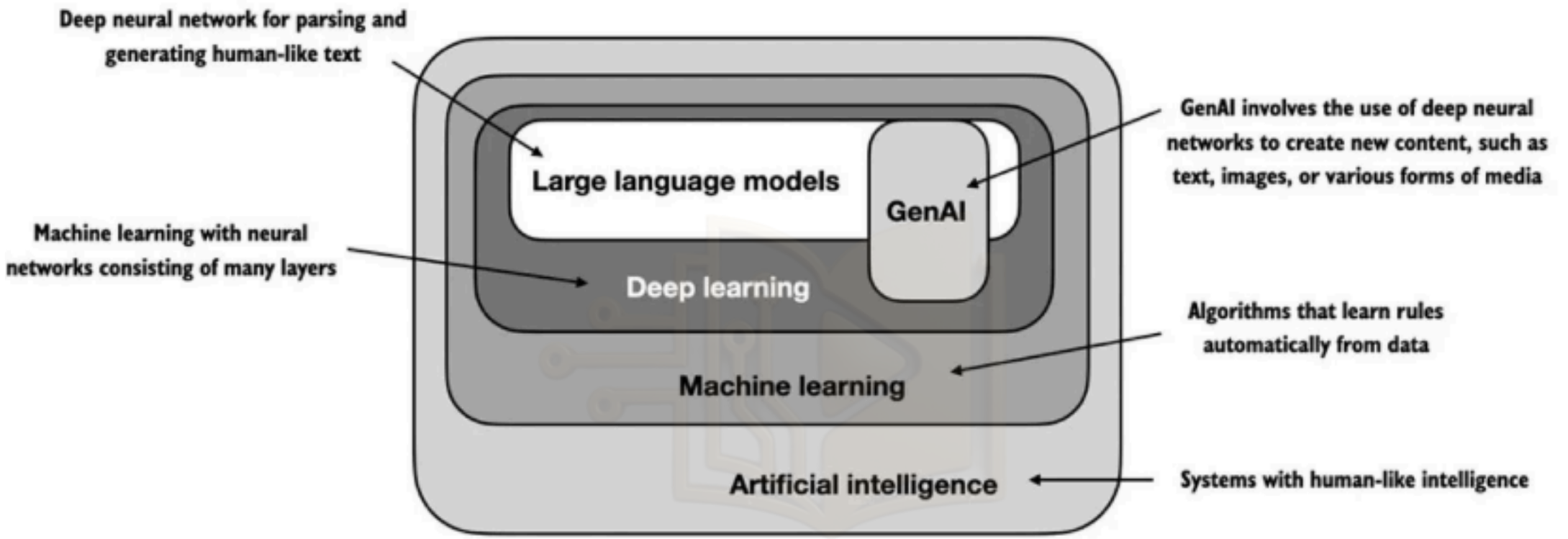
What is Large Language Model?

Available on notes.arpanadhikari7.com.np

• The success behind LLMs can be attributed to the transformer architecture which underpins many LLMs, and the vast amounts of data LLMs are trained on, allowing them to capture a wide variety of linguistic nuances, contexts, and patterns that would be challenging to manually encode

notes.arpanadhikari7.com.np

Available on notes.arpanadhikari7.com.np



By: Basanta Joshi, PhD

notes.arpanadhikari7.com.np

What is Deep Learning?

- **Artificial Neural Networks with "*multiple*" Layers** "Deep Learning

was a rebranding of the modern incarnations of neural nets with more than two layers"

Multiple processing **layers** to learn **REPRESENTATIONS** of (raw) data (with multiple levels of abstraction)



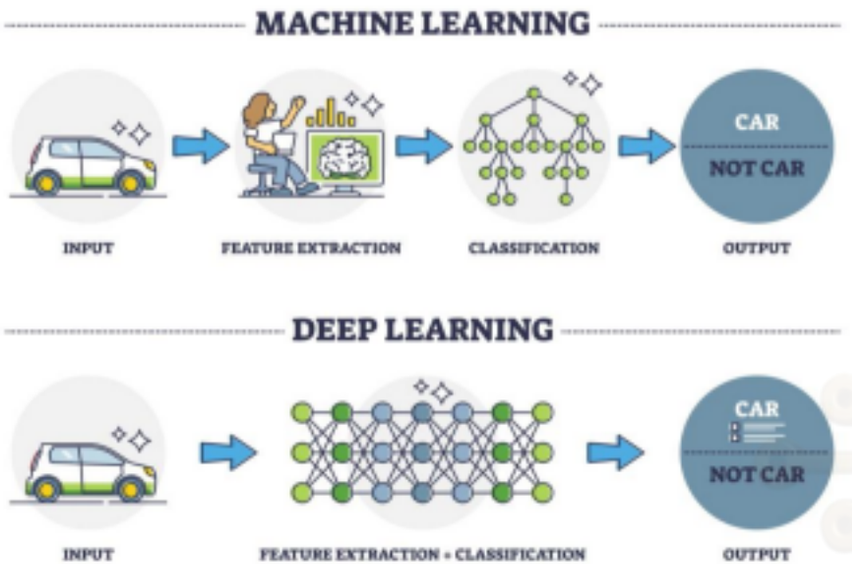
notes.arpanadhikari7.com.np

What is Deep Learning (DL) ?

Available on notes.arpanadhikari7.com.np

- A machine learning subfield of learning **representations** of data. Exceptional effective at **learning patterns**.
- Deep learning algorithms attempt to learn (multiple levels of) representation by using a **hierarchy of multiple layers**
- If you provide the system **tons of information**, it begins to understand it and respond in useful ways. • In ~2010 DL started outperforming other ML techniques
- first in speech and vision, then NLP

notes.arpanadhikari7.com.np

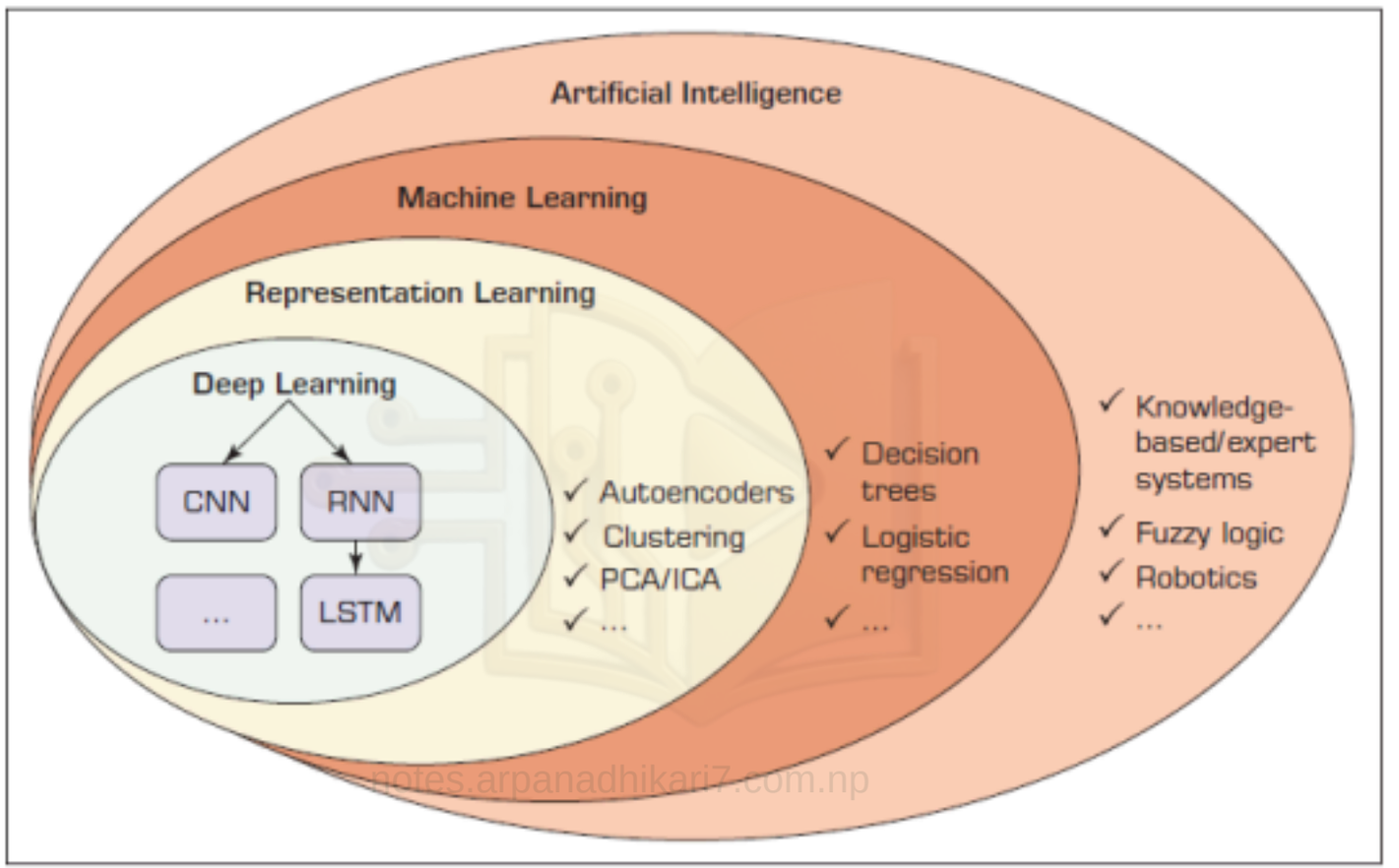




https://www.xenonstack.com/blog/static/public/uploads/media/machine-learning-vs-deep-learning.png

Simple hierarchical relationship diagram

notes.arpanadhikari7.com.np



notes.arpanadhikari7.com.np

FIGURE 6.3 **A Venn Diagram Showing the Placement of Deep Learning within the Overarching AI-Based Learning Methods.**

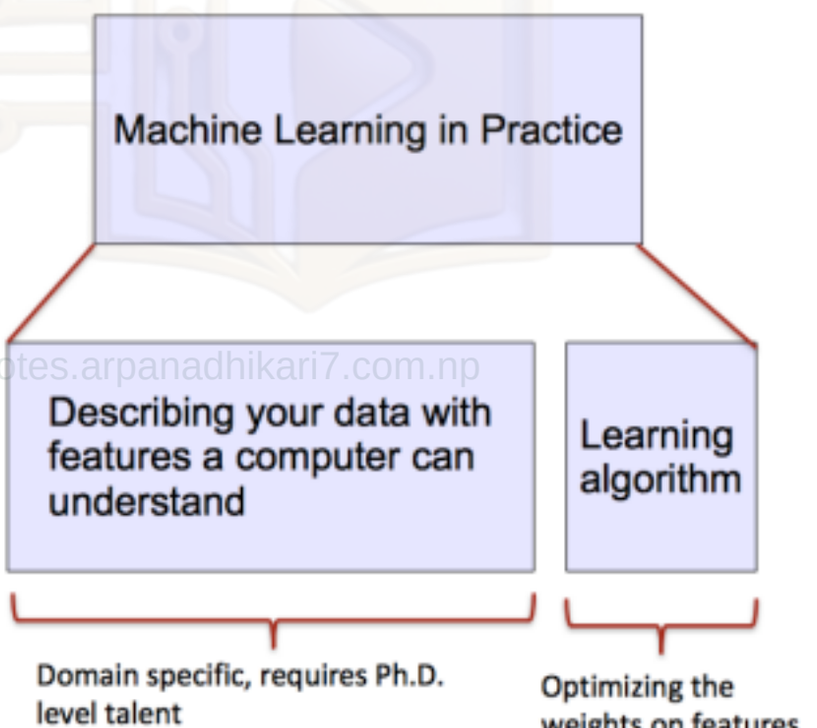
Available on notes.arpanadhikari7.com.np

ML vs. Deep Learning

Most machine learning methods work well because of **human-designed representations** and **input features**

ML becomes just **optimizing weights** to best make a final prediction

Feature	NER
Current Word	✓
Previous Word	✓
Next Word	✓
Current Word Character n-gram	all
Current POS Tag	✓
Surrounding POS Tag Sequence	✓
Current Word Shape	✓
Surrounding Word Shape Sequence	✓
Presence of Word in Left Window	size 4
Presence of Word in Right Window	size 4



notes.arpanadhikari7.com.np

Slide | 9 By: Basanta Joshi, PhD

Deep Learning Vs Classical ML

notes.arpanadhikari7.com.np

Available on notes.arpanadhikari7.com.np

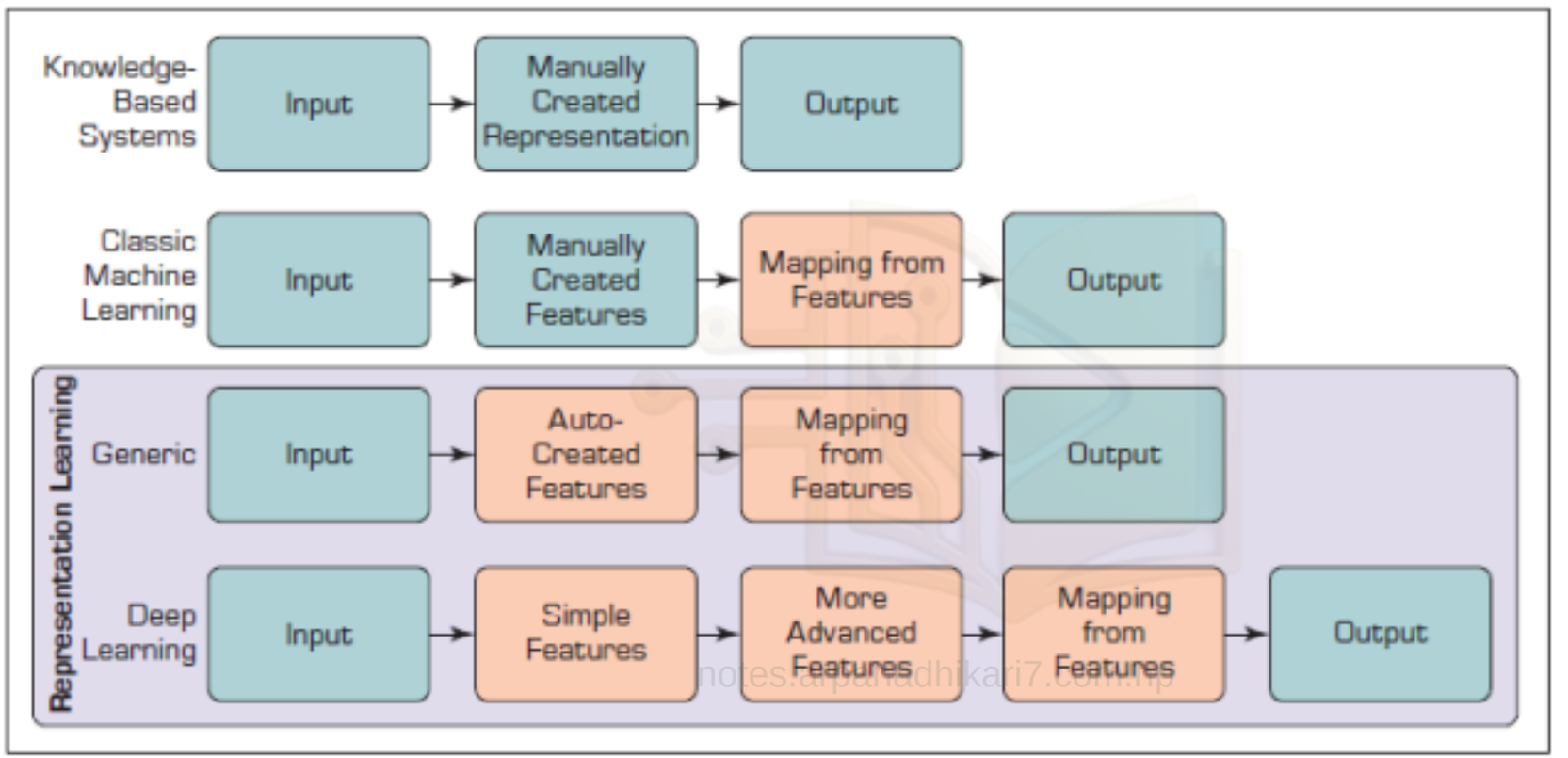


FIGURE 6.4 **Illustration of the Key Differences between Classic Machine-Learning Methods and Representation Learning/Deep Learning (shaded boxes indicate components that are able to learn directly from data).**

Available on notes.arpanadhikari7.com.np

Common Architectures for Deep Learning

- **Convolutional Neural Networks (CNNs)**: Great for image processing.
- **Recurrent Neural Networks (RNNs)**: Used for sequence data like time series and text.
- **Transformers**: Powerful models for language tasks (e.g., ChatGPT is based on one).
- **Autoencoders**: Used for unsupervised learning and dimensionality reduction.
- **GANs (Generative Adversarial Networks)**: Generate new data samples, like deepfakes or art.

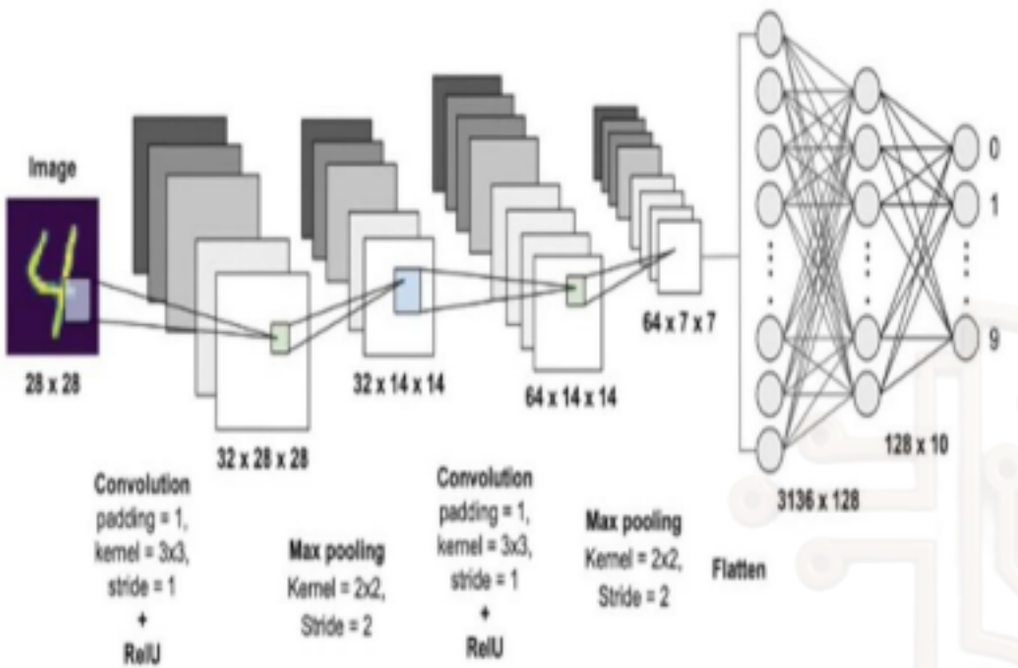
notes.arpanadhikari7.com.np

Available on notes.arpanadhikari7.com.np

CNN – Convolutional Neural Networks

CNNs are a type of deep neural network **specialized for processing grid-like data**, particularly **images**. They capture **spatial hierarchies** through layers that apply **convolutions** instead of fully connected operations.

notes.arpanadhikari7.com.np



Component	Description
Convolutional Layer	Applies filters to detect local patterns (edges, textures, etc.)
Activation (ReLU)	Adds non-linearity
Pooling Layer	Downsamples (reduces spatial dimensions)
Fully Connected Layer	Final classification or regression output
Softmax Layer	Converts output to class probabilities

Available on notes.arpanadhikari7.com.np

CNN (Convolutional Neural Network)

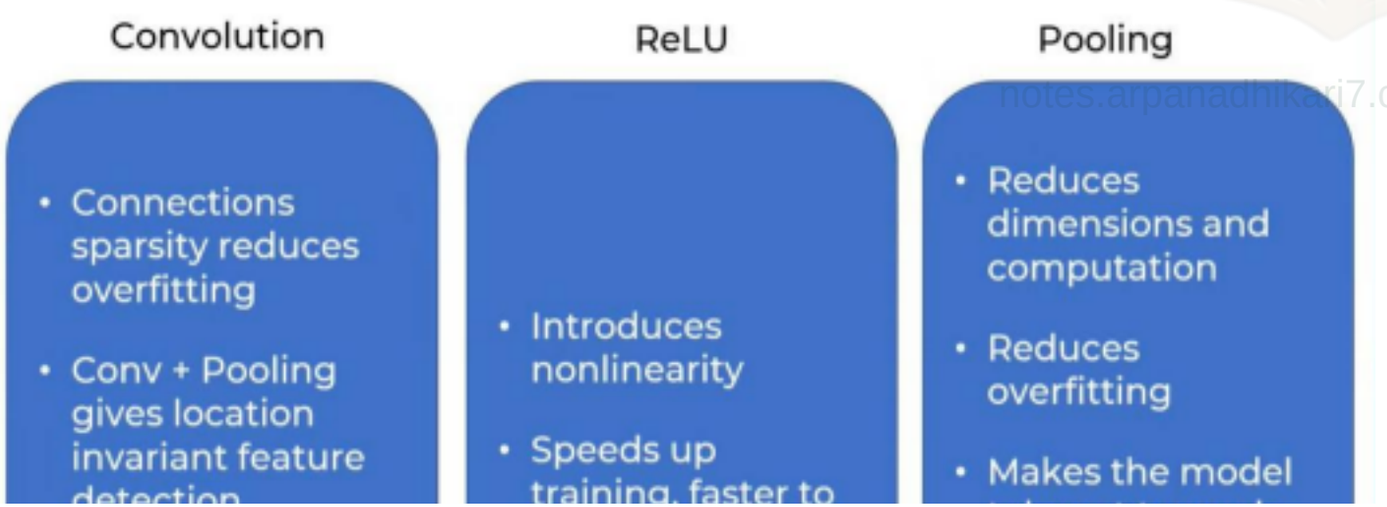
- **Purpose:** Mainly used for processing **grid-like data** such as images, videos, and spatial data.
- **Architecture:** Uses **convolutional layers** to automatically learn spatial hierarchies of features by applying filters/kernels. • **Key Idea:** Extracts **local features** by sliding filters over input data. • **Data Handling:** Processes data with fixed-size input (e.g., images with width × height × channels).
- **Strength:** Excellent at capturing spatial relationships and patterns

Available on notes.arpanadhikari7.com.np

(e.g., edges, textures).

- **Common Applications:** Image classification, object detection, image segmentation, video analysis.

CNN – Convolutional Neural Networks



Benefits of CNN:

- Excellent for image, video, and spatial data.
- Less parameter-heavy than MLPs due to weight sharing.
- Robust to translations and distortions.

Applications:

- Image classification (e.g. ResNet, VGG)
- Object detection (YOLO, Faster R

notes.arpanadhikari7.com.np

Available on notes.arpanadhikari7.com.np

CNN)
•Facial recognition
•Medical imaging
•Style transfer

Zero-Shot Learning (ZSL)

notes.arpanadhikari7.com.np

Zero-shot learning enables models to **recognize classes they've never seen during training**, by using **semantic descriptions** (e.g., attributes or word embeddings).

How it works:

- Learn from seen classes with labeled examples.
- Use semantic embeddings (e.g., word2vec, GloVe, BERT) to **bridge seen and**

Available on notes.arpanadhikari7.com.np

unseen classes.

- During inference, match features of new data to these embeddings. If a model has never seen a “zebra” but knows about "horses" and the **attributes** of a zebra (e.g., striped, 4-legged), it can still classify it correctly.

Benefits:

- Useful when labeled data is scarce for certain classes.
- Enables generalized AI systems.

Few-Shot Learning (FSL)

notes.arpanadhikari7.com.np

Few-shot learning allows models to **learn from a very small number of examples** (e.g., 1–10 samples per class).

Available on notes.arpanadhikari7.com.np

Method	[redacted]
Siamese Networks	Learns to compare similarity between two inputs
Prototypical Networks	Learns a prototype (mean embedding) for each class
Meta-Learning	“Learning to learn” – adapts quickly to new tasks (e.g., MAML)
Transfer Learning	Pretrain on large data, fine-tune on small data

notes.arpanadhikari7.com.np

Benefits:

- Efficient when data collection is expensive (e.g., medical diagnosis).
- Fast adaptation to new tasks.

Relationship to CNN:

Available on notes.arpanadhikari7.com.np

- CNNs often serve as **feature extractors** in FSL and ZSL. The last layers are adapted for few/zero-shot generalization.

RNN (Recurrent Neural Network)

notes.arpanadhikari7.com.np

- **Purpose:** Designed to handle **sequential or time-series data** by maintaining a hidden state that captures information from previous steps.
- **Architecture:** Has loops within the network to pass information from one step to the next.
- **Key Idea:** Captures **temporal dependencies** by processing data sequentially.
- **Data Handling:** Accepts sequences of variable length.
- **Strength:** Models temporal or sequential patterns (e.g., sentences, speech, time series).

Available on notes.arpanadhikari7.com.np

- **Limitation:** Struggles with **long-term dependencies** due to vanishing/exploding gradients.
- **Common Applications:** Language modeling, speech recognition, sequence prediction.

RNN (Recurrent Neural Network)

- A **Recurrent Neural Network (RNN)** is a type of neural network designed to **handle sequential data** (**e.g., text, time series, audio**). Unlike feedforward networks, RNNs have **loops** that allow **information to persist across time steps**.

Why use RNNs?

•To process data where **order matters**.

Available on notes.arpanadhikari7.com.np

•To **retain memory** of previous inputs in the sequence.

How RNNs Work:

At each time step t, RNN takes:

•Current input xt

•Previous hidden state h t−1

$$h_t = \tanh(W_{xh}x_t + W_{hh}h_{t-1} + b_h)$$

$$y_t = W_{hy}h_t + b_y$$

Where:

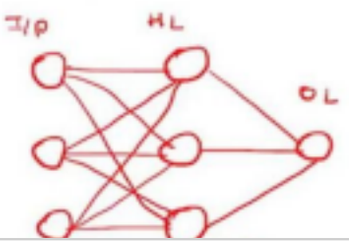
- W_{xh}: Input → Hidden weights
- W_{hh}: Hidden → Hidden (recurrent) weights
- W_{hy}: Hidden → Output weights

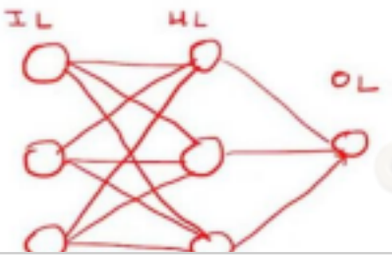
notes.arpanadhikari7.com.np

Available on notes.arpanadhikari7.com.np

Recurrent Neural Network

- RNN is a feedback network in which signal can flow in both directions using close loop.
- **RNN** is a kind of ANN mainly used in speech recognition & natural language processing (NLP) to predict independent next word of sentence.
- In RNN, hidden layer has a memory to remembers information about sequence that what has been calculated.

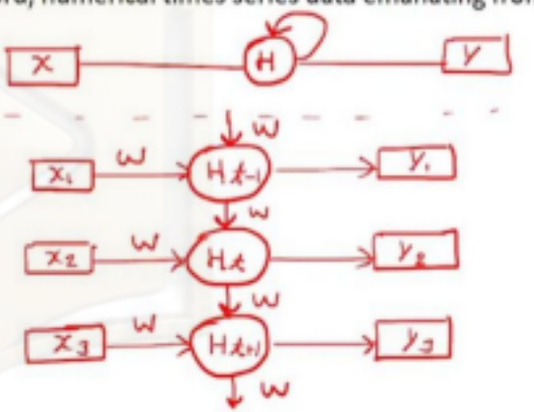




notes.arpanadhikari7.com.np

Recurrent Neural Network

- In RNN, output from previous step are fed as input to current step unlike feed forward network in which input and output are independent to each other.
- used to time series prediction to remember inf through time & extend pixel neighborhood.
- It use to identify patterns in data sequences, such as text, genomes, handwriting, the spoken word, numerical times series data emanating from sensors, stock markets.



Available on notes.arpanadhikari7.com.np

Application and Comparison

- **Natural Language Processing (NLP):**
- Sentiment analysis
- Machine translation
- Text generation
- **Speech recognition**
- **Time series forecasting**
- **Music generation**

notes.arpanadhikari7.com.np

Variant		Best for
Vanilla RNN	Basic RNN with short-term memory	Simple sequences
LSTM (Long Short-Term Memory)	Handles long-term dependencies via gates	Long sequences
GRU (Gated Recurrent Unit)	Simplified version of LSTM, fewer parameters	Faster training
Bidirectional RNN	Processes sequence in both directions	NLP tasks
Encoder-Decoder (Seq2Seq)	Used in translation, summarization	Input/output sequences

notes.arpanadhikari7.com.np

Challenges of RNNs:

Available on notes.arpanadhikari7.com.np

Solutions: Use LSTM, GRU, attention mechanisms, or switch to Transformers.

Benefits of RNNs:

•Handles variable-length sequences •Captures temporal dynamics •Shares parameters across time

Limitations:

•Hard to train on **long sequences** •Sequential nature limits **parallelization**

•Replaced in many applications by **Transformers**

notes.arpanadhikari7.com.np

Problem	Explanation
Vanishing gradient	Hard to learn long-term dependencies
Exploding gradient	Large updates destabilize learning

Available on notes.arpanadhikari7.com.np

Slow training	⚠
	rallelize across time steps easily
Memory bottleneck	Hidden states may not capture all relevant info

LSTM (Long Short-Term Memory)

notes.arpanadhikari7.com.np

- **Purpose:** A special kind of RNN designed to overcome RNN's vanishing gradient problem, capable of learning **long-term dependencies**. • **Architecture:** Uses **gates** (input, forget, output) to control information flow and maintain memory over long sequences.

- **Key Idea:** Can selectively **remember or forget** information using gated mechanisms.
- **Data Handling:** Like RNN, works with sequential data but better at preserving context over many time steps.
- **Strength:** Excellent for long sequences where context from far earlier in the sequence matters.
- **Common Applications:** Machine translation, speech synthesis, time-series forecasting, text generation.

Slide | 24 By: Basanta Joshi, PhD
notes.arpanadhikari7.com.np

What is Attention?

Attention mechanisms in neural networks allow models to **focus on the most relevant parts** of the input when generating

Available on notes.arpanadhikari7.com.np

outputs—mimicking how humans pay selective attention.

- **Why is Attention Important?**
- Solves the **bottleneck** problem in sequence models (e.g., RNNs, LSTMs), which try to compress all input into a single vector.

Key idea: Instead of processing all input equally, attention **weights** input elements **differently**, depending on their importance to the task.

•Enables **better long-range dependency handling**.

•Forms the **foundation of Transformers**, which have replaced RNNs in many NLP tasks.

notes.arpanadhikari7.com.np

Attention Mechanism

Available on notes.arpanadhikari7.com.np

Type	
Additive Attention	Computes score using a feedforward network (used in early models like Bahdanau attention)
Dot-product Attention	Simpler, faster; uses dot products between Q and K
Self-Attention	Query, Key, and Value come from the **same** input sequence
Cross-Attention	Q comes from one source, K and V from another (used in encoder-decoder models)
Multi-Head Attention	Runs multiple self-attention mechanisms in parallel and concatenates outputs

1. Basic Attention Mechanism

Core components:

- **Query (Q)**: What we're looking for.

Available on notes.arpanadhikari7.com.np

- **Key (K)**: What each input element offers.
- **Value (V)**: What we actually use from the input.



- **Scoring and Weighting:**

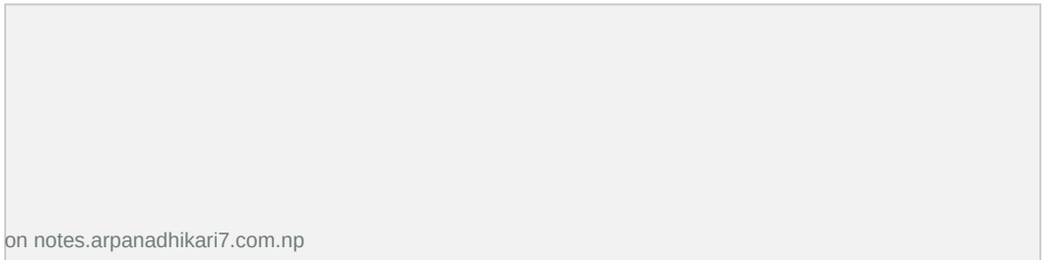
•The softmax gives **attention weights**.

•These weights are used to compute a **weighted sum of values (V)**.

Self-Attention (used in Transformers)

notes.arpanadhikari7.com.np

Self-attention relates **each position in a sequence to every other position**, allowing the model to understand



Available on notes.arpanadhikari7.com.np

context and relationships.

Let’s say we input a sentence:

"The cat sat on the mat"
•Self-attention lets the model understand that "cat" is the subject and "sat" is the verb, even if they are several words apart.

notes.arpanadhikari7.com.np

Multi-Head Attention

Available on notes.arpanadhikari7.com.np

Instead of one attention computation, the model computes multiple "heads": MultiHead(Q,K,V)=Concat(head1,...,headh)WO
Each head learns different relationships in the data.

Used in:

•Transformer architecture (BERT, GPT, T5) •Vision Transformers (ViT)
•Machine Translation
•Speech & time-series modeling

Task
Machine Translati
Text Summar
Image Captioni
Speech Recogni

notes.arpanadhikari7.com.np

Benefit	

Available on notes.arpanadhikari7.com.np

	ters
Focus on relevant input	Enhances model interpretability & performance
Handles long sequences	Better than RNNs/LSTMs for long-range dependencies
Parallelizable	Especially in Transformers, unlike RNNs
General-purpose	Works in NLP, vision, audio, graphs, etc.

What Is a Transformer?

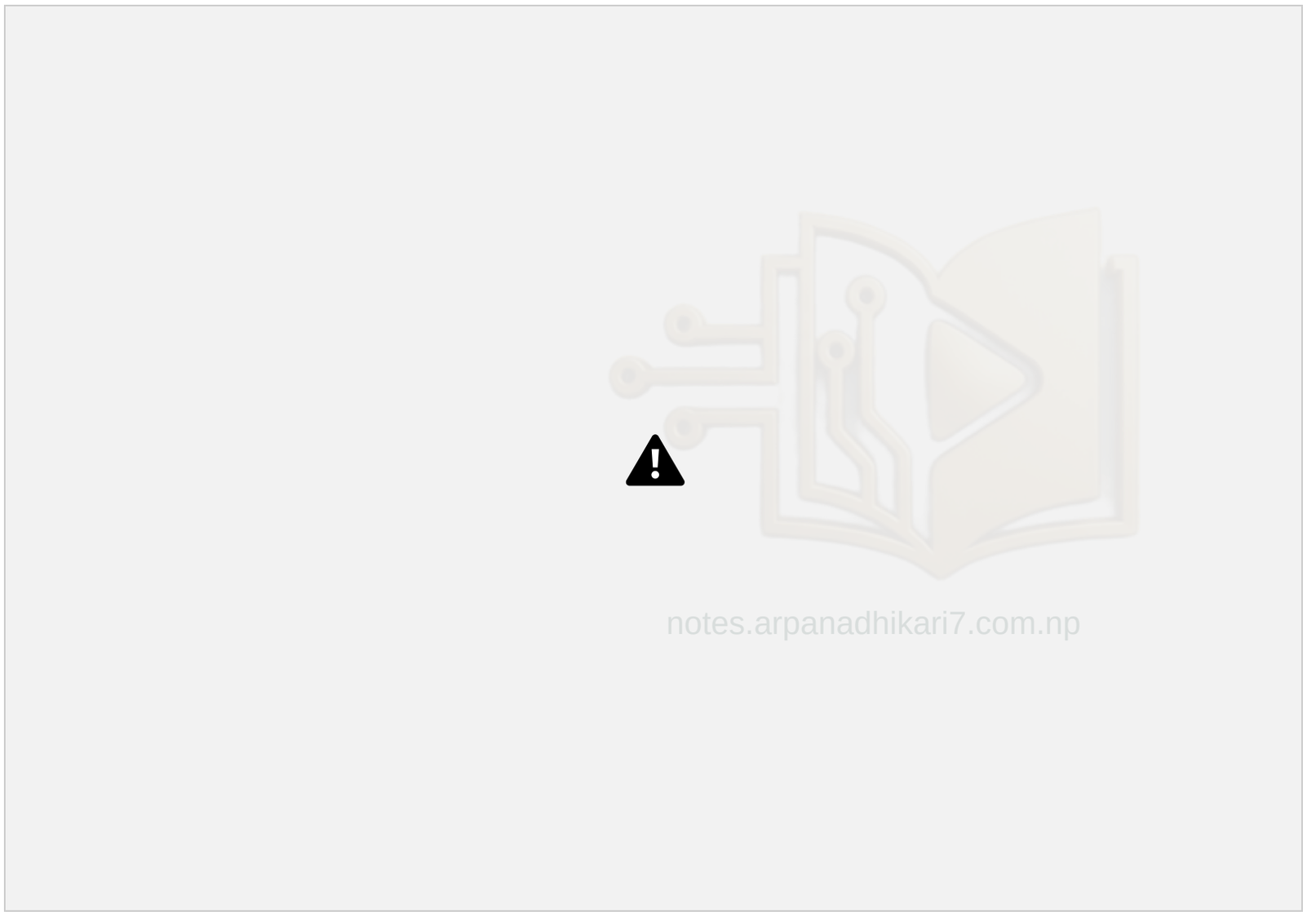
notes.arpanadhikari7.com.np

- The **Transformer** architecture was introduced in the 2017 paper **"Attention is All You Need"** by Vaswani et al. It discards traditional recurrent structures (like RNNs or LSTMs) and instead relies entirely on **attention mechanisms** to model relationships between elements in a sequence.

- Transformers are a type of neural network architecture that have revolutionized the field of deep learning, particularly in **natural language processing (NLP)**, but also in areas like computer vision, time-series analysis, and more.

Transformer Architecture

notes.arpanadhikari7.com.np



notes.arpanadhikari7.com.np

Key Components of a Transformer

1. Input Embedding + Positional Encoding

•Converts tokens (like words) into vectors.

•Since transformers don't have recurrence or convolution, they add **positional encoding** to capture the order of tokens.

2. Multi-Head Self-Attention

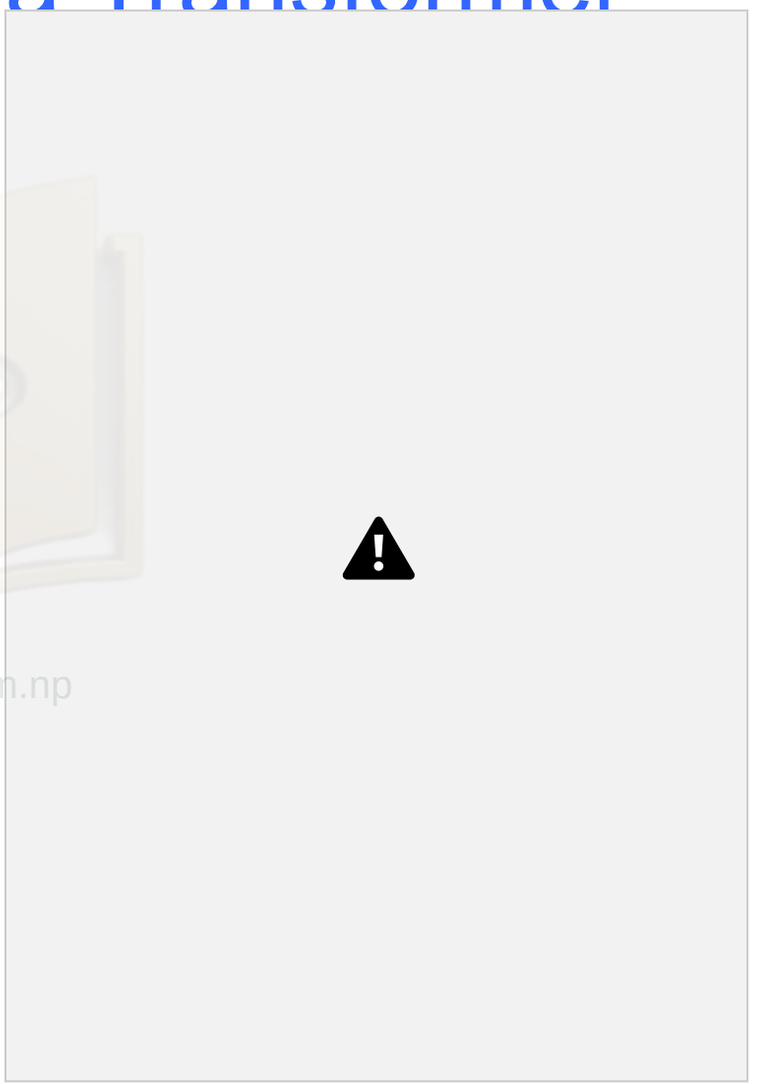
•Allows the model to **weigh the importance** of other tokens in the sequence when encoding a particular token.

•Multiple attention heads allow the model to focus on different relationships.

3. Feed-Forward Neural Network (FFN)

•A position-wise fully connected network applied to each position's output from the attention layer.

4. Residual Connections + Layer Normalization



notes.arpanadhikari7.com.np

•Helps in training deep networks by preventing vanishing gradients.

5. Encoder-Decoder Structure (for sequence-to-sequence tasks) •**Encoder**: Processes the input sequence.

•**Decoder**: Generates the output sequence, using encoder outputs and previously generated outputs.

Transformer Variants

•**BERT (Bidirectional Encoder Representations from Transformers)**: Uses only the encoder part, ideal for classification, Q&A, etc.

•**GPT (Generative Pre-trained Transformer)**: Uses only the decoder part (unidirectional), ideal for text generation.

notes.arpanadhikari7.com.np

•**T5, XLNet, RoBERTa, LLaMA, etc.**: Variants with different improvements and training schemes.

Applications

•Machine translation

•Text summarization

•Sentiment analysis

•Text generation (e.g., ChatGPT)

•Image classification (Vision Transformers) **•Time-series forecasting**

Why Are Transformers Powerful?

Model Name	Organiz ati on	Use Case
GPT-4 / GPT-3.5	OpenAI	General-purpose text generation
BERT (not generative)	Google	Used for understanding, not generating text
5	Google	Text-to-text generation tasks
LaMA	Meta	Open-source generative model
aude	Anthropic	Conversational AI

•**Parallelizable**: Unlike RNNs, transformers process sequences all at once (not token by token). •**Scalable**: Can be trained on massive datasets with GPUs/TPUs.
•**Generalizable**: Foundation for large pre-trained models (e.g., GPT, BERT) that can be fine-tuned for many tasks.

BERT vs GPT

notes.arpanadhikari7.com.np

- BERT, which is built upon the original transformer's encoder submodule, differs in its training approach from GPT
- While GPT is designed for generative tasks, BERT and its variants specialize in masked word prediction, where the model predicts masked

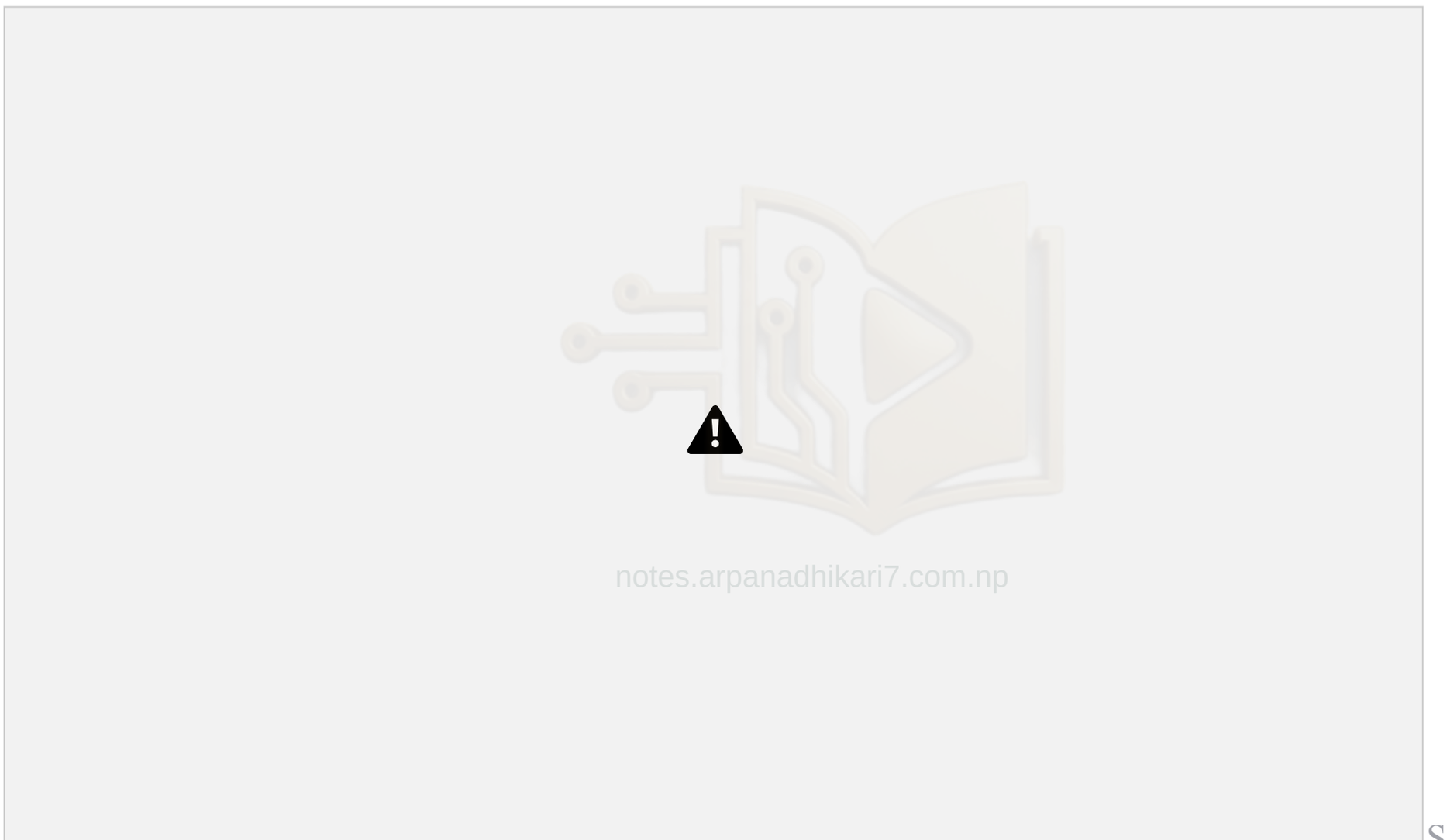
or hidden words in a given sentence

- This unique training strategy equips BERT with strengths in text classification tasks, including sentiment prediction and document categorization. As an application of its capabilities, as of this writing, Twitter uses BERT to detect toxic content.

Slide | 33 By: Basanta Joshi, PhD
notes.arpanadhikari7.com.np

BERT vs GPT

Available on notes.arpanadhikari7.com.np



Available on notes.arpanadhikari7.com.np

Basanta Joshi, PhD

Tokenization & Embeddings

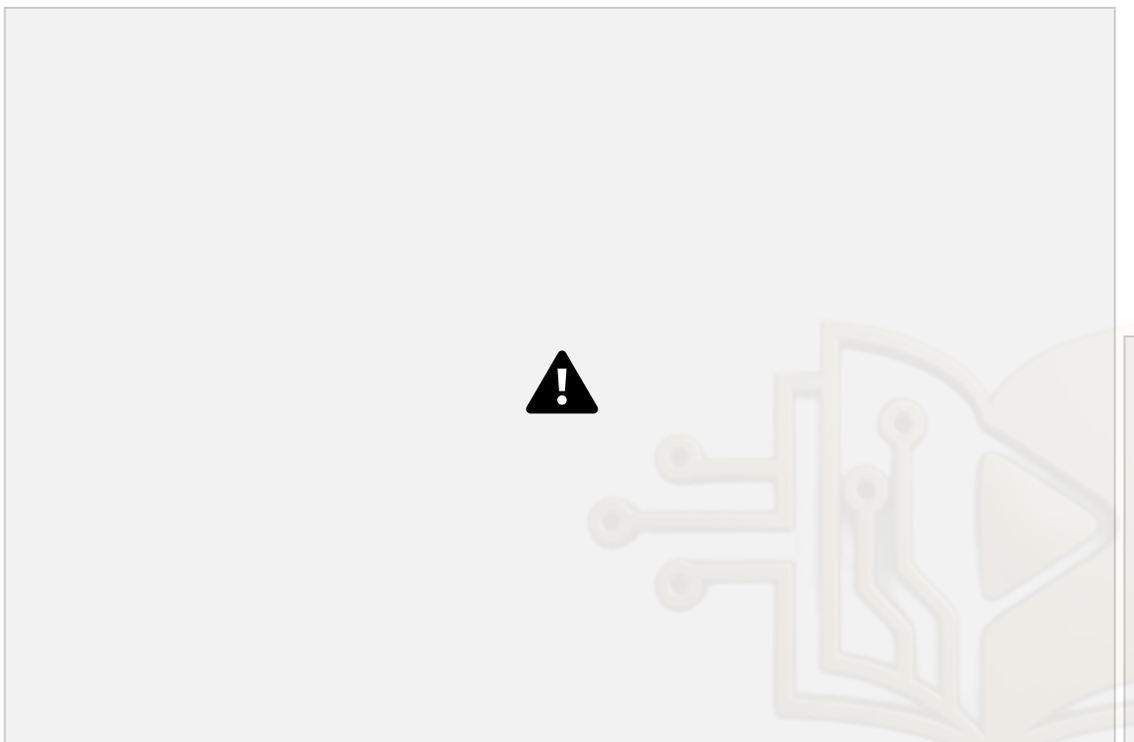
Aspect	Tokenization	Embeddings
Purpose	Splits text into smaller units (tokens)	Converts tokens into numerical vectors
Process	Converts text like "Hello, world!" into tokens ["Hello", ",", "world", "!"] Maps tokens to high dimensional vectors (e.g., Word2Vec, BERT)	
Role	Prepares text for further processing	Provides dense representations capturing semantic meaning

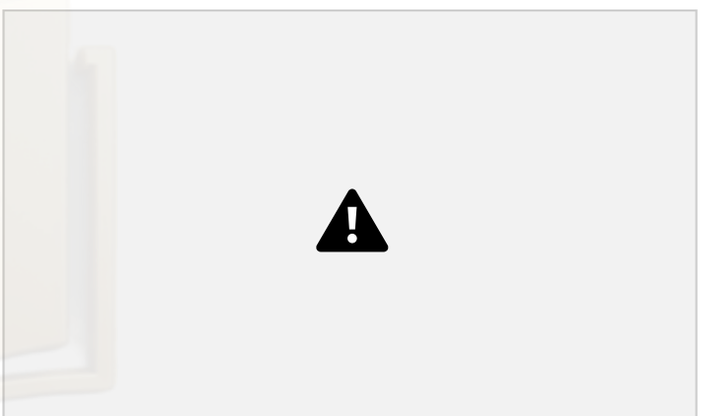
Available on notes.arpanadhikari7.com.np

Granularity	Works at word, subword, or character level Works at vector representation level	
Example	Output ["Hello", ",", "world", "!"]	Vectors like [0.5, 0.1, …, 0.3]

notes.arpanadhikari7.com.np

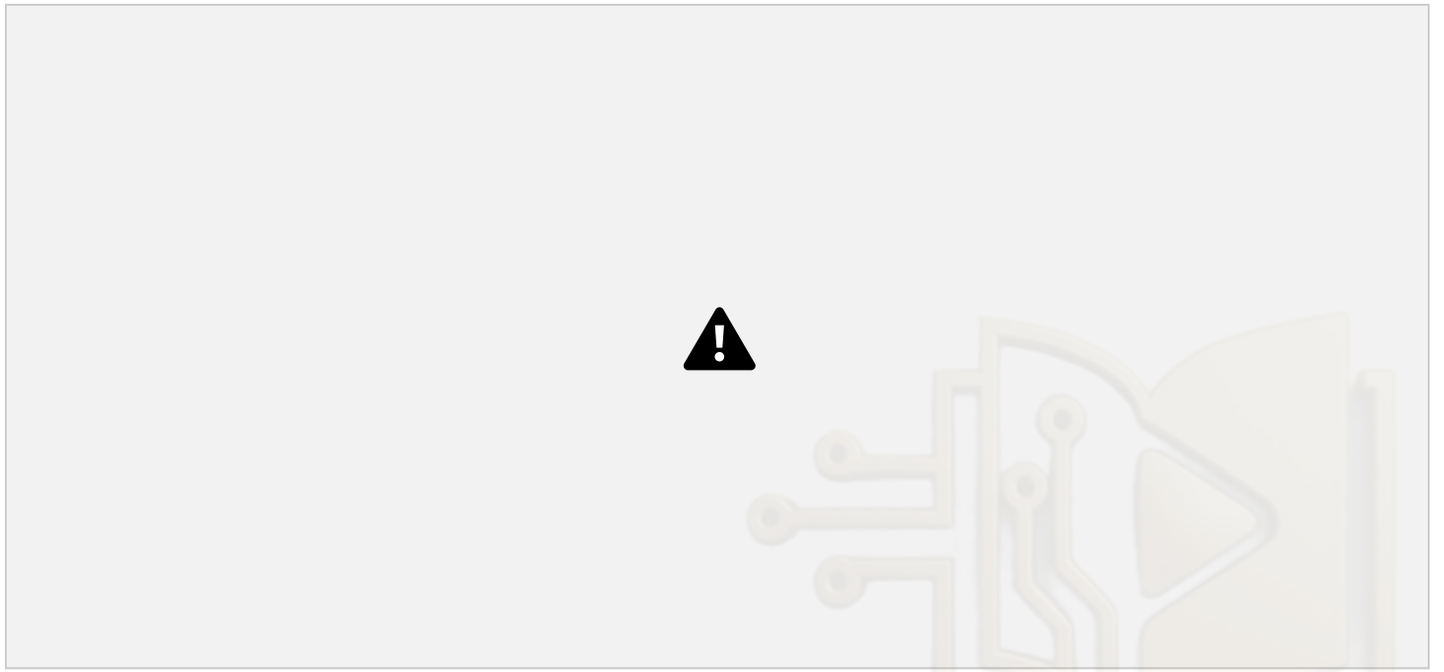
Available on notes.arpanadhikari7.com.np





notes.arpanadhikari7.com.np

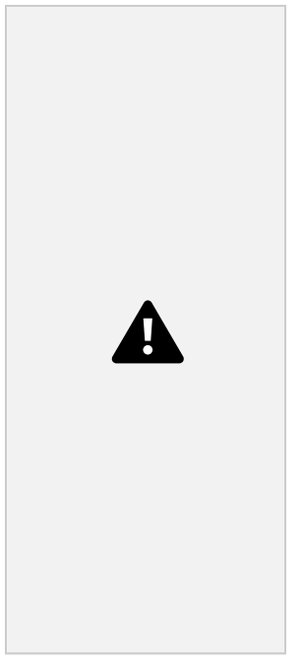
Available on notes.arpanadhikari7.com.np





notes.arpanadhikari7.com.np

Available on notes.arpanadhikari7.com.np



notes.arpanadhikari7.com.np

What Is Transfer Learning?

Transfer Learning is a powerful technique in **neural and deep learning** where a model trained on one task is reused (wholly or in part) for a different but related task.

Transfer learning leverages **knowledge from a pre-trained model** to solve a new problem, often when you have **limited labeled data** for the new task.

For example:

•A model trained on **ImageNet** to classify 1000 objects can be reused to classify **medical images** or **wildlife species**.

How It Works

Here's a high-level view of how transfer learning is used in practice:

1. Pretraining Phase

- A deep network (e.g., CNN, Transformer) is trained on a **large dataset** (e.g., ImageNet for images, Wikipedia for NLP).
- The model learns general-purpose features like edges, shapes, sentence structure, or semantic relationships.

2. Transfer Phase (Fine-Tuning or Feature Extraction)

- You take the pre-trained model and:
  - **Freeze** some of the earlier layers (so they don't change).
  - **Fine-tune** the later layers using your **task-specific dataset**.
  - Or add new output layers suited to your task.

notes.arpanadhikari7.com.np

Common Transfer Learning Strategies

Strategy	Description

Available on notes.arpanadhikari7.com.np

Feature Extraction	Use the pre-trained model as a fixed feature extractor (freeze all layers except the classifier).
Fine-Tuning	Unfreeze some or all layers of the pre-trained model and retrain them on new data.
Partial Transfer	Transfer only a subset of weights/layers from the source model.

Popular Models for Transfer Learning

For Images:

•**ResNet**, **VGG**, **Inception**, **EfficientNet** (trained on ImageNet)

•Use in medical imaging, face recognition, etc.

For NLP:

•**BERT**, **GPT**, **RoBERTa**, **T5**

•Pretrained on massive corpora → fine-tuned for sentiment analysis, question answering, summarization, etc.

notes.arpanadhikari7.com.np

Why Transfer Learning Works

Deep neural networks tend to learn **general features** in the early layers and **task-specific features** in the later ones. By reusing the general features, transfer learning enables: •**Faster training**

•**Better accuracy with less data**

•**Reduced need for large computational resources**

Available on notes.arpanadhikari7.com.np

Pretraining vs Finetuning

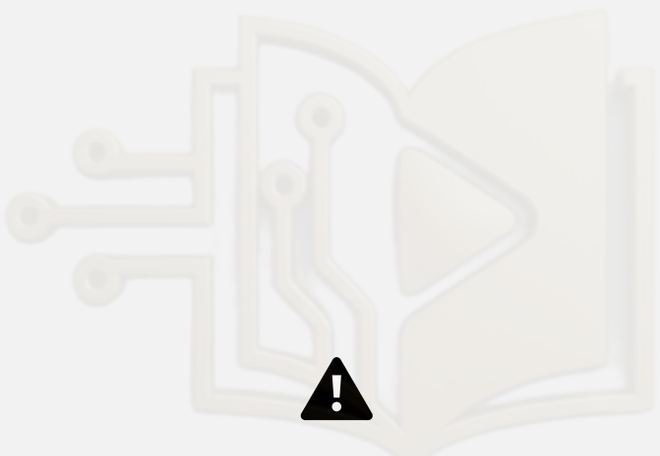
- The general process of creating an LLM includes pretraining and finetuning. The term "pre" in "pretraining" refers to the initial phase where a model like an LLM is trained on a large, diverse dataset to develop a broad understanding of language
- This pretrained model then serves as a foundational resource that can be further refined through finetuning, a process where the model is specifically trained on a narrower dataset that is more specific to particular tasks or domains.

Available on notes.arpanadhikari7.com.np

Slide | 50 By: Basanta Joshi, PhD

Pretraining vs Finetuning

notes.arpanadhikari7.com.np



notes.arpanadhikari7.com.np

Instruction-finetuning and Finetuning for classification tasks

- The two most popular categories of finetuning LLMs include instruction finetuning and finetuning for classification tasks
- In instruction-finetuning, the labeled dataset consists of instruction and answer pairs, such as a query to translate a text accompanied by the correctly translated text. In classification finetuning, the labeled dataset consists of texts and associated class labels, for example, emails associated with spam and non-spam labels

Available on notes.arpanadhikari7.com.np

Cost of Pretraining LLMs

notes.arpanadhikari7.com.np

- The pretrained nature of these models makes them incredibly versatile for further finetuning on downstream tasks, which is why they are also known as base or foundation models
- Architectures such as GPT-3 are also significantly larger than the original transformer model. For instance, the original transformer repeated the encoder and decoder blocks six times. GPT-3 has 96

Available on notes.arpanadhikari7.com.np

transformer layers and 175 billion parameters in total.

- Pretraining LLMs requires access to significant resources and is very expensive. For example, the GPT-3 pretraining cost is estimated to be $4.6 million in terms of cloud computing credits

Slide | 53 By: Basanta Joshi, PhD

Pretrained LLMs

notes.arpanadhikari7.com.np

- The good news is that many pretrained LLMs, available as open-source models, can be used as general purpose tools to write, extract, and edit texts that were not part of the training data
- Also, LLMs can be finetuned on specific tasks with relatively smaller

Available on notes.arpanadhikari7.com.np

datasets, reducing the computational resources needed and improving performance on the specific task

Slide | 54 By: Basanta Joshi, PhD
notes.arpanadhikari7.com.np

Comparison

GPT OpenAI Decoder-Only	Decoder-Only	GPT-4o /GPT-4 Turbo ~1.8 Trillion	1.8 Trillion	Input: $2.50 Output: $10.00
BERT Google	Encoder-Only	BERT-Large	340 Million	N/A (Used for embedding/understanding; non generative, self-hosted)
T5 Google	Encoder-Decoder	T5-11B / Flan-T5 XXL	11 Billion	N/A (Open-source, self-hosted deployment)
LLaMA Meta	Decoder-Only	LLaMA 3.1 / 3.3	8B, 70B, and 405B parameters	Token API: Free (Self-hosted) Hosted APIs: ~$0.60 - $2.00 / 1M tokens
Claude Anthropic	Decoder-Only	Claude 3.5 Sonnet / Opus	Hundreds of Billions	Input: $3.00Output: $15.00

notes.arpanadhikari7.com.np

Available on notes.arpanadhikari7.com.np

Gemini Google	Native Multimodal Decoder	Gemini 1.5 Pro / 2.0 Flash	Hundreds of Billions	Input: $1.25 – $2.00 Output: $10.00 – $12.00

Autoregressive Models

- Compared to the original transformer architecture we covered in section, Using LLMs for different tasks, the general GPT architecture is relatively simple
- Since decoder-style models like GPT generate text by predicting text one word at a time, they are considered a type of autoregressive model • Autoregressive models incorporate their previous outputs as inputs for future predictions. Consequently, in GPT, each new word is chosen based on the sequence that precedes it, which improves coherence of the

Available on notes.arpanadhikari7.com.np

resulting text

Slide | 56 By: Basanta Joshi, PhD

Autoregressive Models

notes.arpanadhikari7.com.np

notes.arpanadhikari7.com.np

Available on notes.arpanadhikari7.com.np

Building an LLM

notes.arpanadhikari7.com.np

notes.arpanadhikari7.com.np

Available on notes.arpanadhikari7.com.np

Thank you !!!

notes.arpanadhikari7.com.np

Available on notes.arpanadhikari7.com.np